

**INDIAN INSTITUTE OF INFORMATION
TECHNOLOGY DESIGN AND
MANUFACTURING, KURNOOL**



Project Report

**AHB-Lite Memory Controller
(RTL to GDSII Implementation)**

VLSI System Design Practice - EC 307

**A. Gayatri
123EC0044**

Academic Year: 2025-26

Contents Overview

1. Aim
2. Apparatus
3. Theory
4. Verilog Code (Design)
5. Verilog Testbench Code
6. Simulation Procedure (Genus & Innovus)
 - 6.1. Functional Simulation
 - 6.2. Synthesis using Cadence Genus
 - 6.3. Physical Design using Cadence Innovus
7. Results
8. Conclusion

1. Aim

To design, simulate, and implement an **AHB-Lite Memory Controller** using Verilog HDL and perform physical design steps using **Cadence Genus and Innovus** tools.

2. Apparatus

Cadence Genus, Cadence Innovus , ModelSim / Icarus Verilog Verilog HDL Linux Environment

3. Theory

The **AHB-Lite protocol** is a simplified version of ARM's Advanced High-Performance Bus (AHB) designed for single-master systems. It supports burst transfers, pipelined operations, and single-clock-edge transitions, enabling high-performance communication between the processor core and on-chip peripherals. Unlike full AHB, AHB-Lite removes multi-master support, which simplifies the bus architecture and reduces hardware complexity.

AHB-Lite Architecture Overview

An AHB-Lite system typically consists of three main components:

- **Master:** Initiates all bus transfers by providing control signals such as address, transfer type, and write/read information.
- **Slave:** Responds to requests from the master. In this project, the slave is implemented as a memory controller.
- **Decoder and Multiplexer:** Responsible for address decoding and data routing between the master and the appropriate slave device.

Memory Controller Functionality

A **memory controller** serves as an interface between the AHB-Lite bus and the internal memory array. It ensures proper data transfer timing, manages read/write operations, and maintains protocol compliance. During a write operation, the controller accepts data from the master and stores it in memory. During a read operation, it retrieves data from memory and provides it to the master through the HRDATA bus.

The design ensures correct handling of:

- **Address decoding:** Mapping address bits to specific memory locations.
- **Data synchronization:** Ensuring data is captured on the correct clock edge.
- **Bus readiness:** Managing HREADY and HREADYOUT signals for pipelined transfers.
- **Response signaling:** Returning OKAY response ($HRESP = 2'b00$) to indicate successful transactions.

VLSI Design Flow

The complete VLSI design flow followed in this project includes:

1. **RTL Design, Simulation, and Verification** using **Xilinx Vivado** — designing and functionally verifying the AHB-Lite memory controller.
2. **Synthesis** using **Cadence Genus** — converting RTL into a gate-level netlist optimized for timing, area, and power.
3. **Physical Design (PnR)** using **Cadence Innovus** — performing floor-planning, placement, routing, and clock tree synthesis.
4. **Physical Verification** — performing DRC/LVS checks and generating the final **GDSII** layout for fabrication.

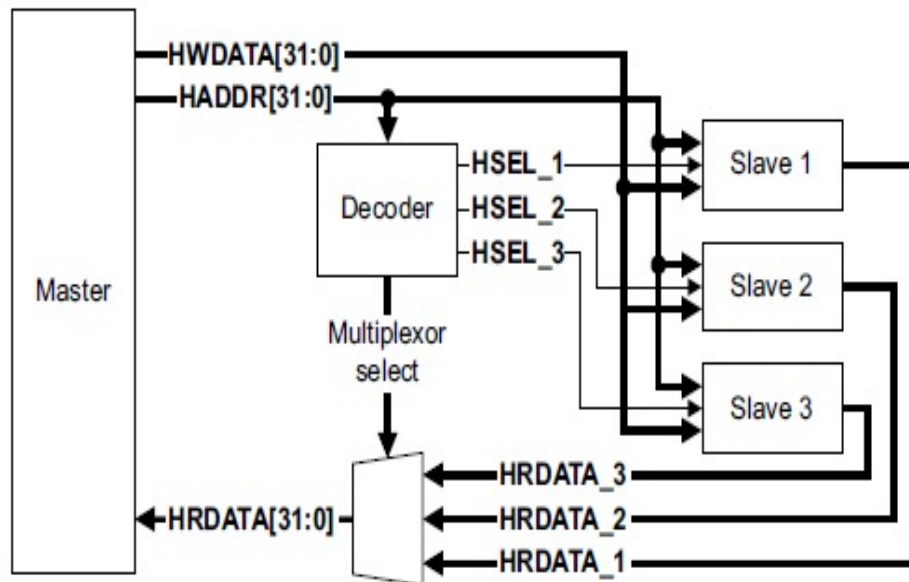


Figure 1: Structure of AMBA - AHB.

4. Verilog Code (Design)

Verilog RTL Code

```
//=====
// AHB-Lite Memory Controller RTL Design
//=====
module ahb_lite_mem_ctrl(
    input  wire      HCLK,
    input  wire      HRESETn,
    input  wire      HSEL,
    input  wire [31:0] HADDR,
    input  wire [1:0] HTRANS,
    input  wire      HWRITE,
    input  wire [31:0] HWDATA,
    output reg [31:0] HRDATA,
    input  wire      HREADY,
    output reg      HREADYOUT,
    output wire [1:0] HRESP
);
    assign HRESP = 2'b00; // OKAY response
    reg [31:0] mem [0:255];

    always @(posedge HCLK or negedge HRESETn) begin
        if (!HRESETn) begin
            HRDATA <= 32'b0;
            HREADYOUT <= 1'b1;
        end
        else if (HSEL && HREADY && HTRANS[1]) begin
            if (HWRITE)
                mem[HADDR[9:2]] <= HWDATA;
            else
                HRDATA <= mem[HADDR[9:2]];
            HREADYOUT <= 1'b1;
        end
    end
end
endmodule
```

5. Verilog Testbench Code

Verilog Testbench

```
module tb_ahb_lite_mem_ctrl();
    reg HCLK, HRESETn, HSEL, HWRITE, HREADY;
    reg [31:0] HADDR, HWDATA;
    reg [1:0] HTRANS;
    wire [31:0] HRDATA;
    wire HREADYOUT;
    wire [1:0] HRESP;
    // Instantiate the DUT (Design Under Test)
    ahb_lite_mem_ctrl uut (
        .HCLK(HCLK),
        .HRESETn(HRESETn),
        .HSEL(HSEL),
        .HADDR(HADDR),
        .HTRANS(HTRANS),
        .HWRITE(HWRITE),
        .HWDATA(HWDATA),
        .HRDATA(HRDATA),
        .HREADY(HREADY),
        .HREADYOUT(HREADYOUT),
        .HRESP(HRESP)
    ); // Clock generation
    always #5 HCLK = ~HCLK;
    // Test sequence
    initial begin
        HCLK = 0;
        HRESETn = 0;
        HSEL = 0;
        HWRITE = 0;
        HREADY = 1;
        HADDR = 0;
        HWDATA = 0;
        HTRANS = 2'b00;
        #10 HRESETn = 1;
        // Write Operation
        HSEL = 1;
        HWRITE = 1;
        HTRANS = 2'b10;
        HADDR = 32'h00000004;
        HWDATA = 32'hA5A5A5A5;
        #10; // Read Operation
        HWRITE = 0;
        HADDR = 32'h00000004;
        #10;
        #10 $finish;
    end
endmodule
```

```
# =====
# Timing and Design Constraints for AHB-Lite Memory Controller
# =====

# Create main clock (100 MHz = 10 ns period)
create_clock -name HCLK -period 10 [get_ports HCLK]

# Input and output delay assumptions
set_input_delay 1 -clock HCLK [get_ports {HRESETn HSEL HWRITE HREADY HTRANS[*] HADDR[*] HWDATA[*]}]
set_output_delay 2 -clock HCLK [get_ports {HRDATA[*] HREADYOUT HRESP[*]}]

# Mark asynchronous reset as false path
set_false_path -from [get_ports HRESETn]

# Protect memory from optimization (behavioral array)
set_dont_touch [get_cells mem*]

# Set typical analysis conditions
set_operating_conditions -analysis_type on_chip_variation
```

Figure 2: Constraints.sdc.

```
# =====
# Genus Synthesis Script for AHB-Lite Memory Controller (digital_jyo)
# =====

# Set library search path to your working directory
set_db init_lib_search_path /home/iiitdmk/digital_jyo/

set_db init_lib_search_path /home/install/FOUNDRY/digital/90nm/dig/lib/

# Specify your standard cell library (replace with actual filename)
set_db library slow.lib

# Read RTL
read_hdl {/home/iiitdmk/digital_jyo/ahb_lite_mem_ctrl.v}
# Elaborate design
elaborate

# Read constraints if present
read_sdc /home/iiitdmk/digital_jyo/constraints_input.sdc
# Set synthesis effort
set_db syn_generic_effort medium
set_db syn_map_effort medium
set_db syn_opt_effort medium
# Synthesis steps
syn_generic
syn_map
syn_opt
# Write netlist and constraints
write_hdl > /home/iiitdmk/digital_jyo/ahb_lite_mem_ctrl_netlist.v
write_sdc > /home/iiitdmk/digital_jyo/ahb_lite_mem_ctrl_output.sdc
# Generate reports
report_timing > /home/iiitdmk/digital_jyo/ahb_lite_mem_ctrl_timing.rpt
report_power > /home/iiitdmk/digital_jyo/ahb_lite_mem_ctrl_power.rpt
report_area > /home/iiitdmk/digital_jyo/ahb_lite_mem_ctrl_area.rpt
report_gates > /home/iiitdmk/digital_jyo/ahb_lite_mem_ctrl_gates.rpt

# Optionally open GUI for schematic view
gui_show
```

Figure 3: Run .tcl.

6. Simulation Procedure (Genus and Innovus)

The entire implementation of the **AHB-Lite Memory Controller** follows a standard digital ASIC design flow using the Cadence Tool Suite. The design process includes functional simulation, logic synthesis, and physical design.

1. Functional Simulation (Using INCISIVE)

Functional simulation ensures that the Verilog code performs as expected before moving into synthesis.

Steps:

1. Initialize the terminal and Cadence license environment using:

```
csh
source /home/install/cshrc
```

2. Launch the simulator:

```
nclaunch -new
```

3. Create a new working library and link it through a `cds.lib` file.
4. Add the Verilog design and testbench files to the project.
5. Run the following simulation stages:
 - **Compile:** To check syntax and compile HDL files.
 - **Elaborate:** To build hierarchy between DUT and testbench.
 - **Simulate:** To observe waveforms and verify read/write operations.
6. View simulation results in the waveform window to ensure correct signal transitions.

2. Synthesis using Cadence Genus

Synthesis converts the RTL-level Verilog description into a gate-level netlist mapped to standard cells.

Inputs:

- RTL Verilog files (`.v`)

- Standard Cell Library (.lib)
- Constraint file (.sdc)

Outputs:

- Gate-Level Netlist
- Timing, Power, and Area Reports

Procedure:

1. Start Genus from the terminal using:

```
genus -f rc_script.tcl
```

2. The synthesis process includes:

- **Translation:** Converts RTL to an intermediate representation.
- **Mapping:** Maps logic to standard cells from the library.
- **Optimization:** Minimizes area and power while meeting timing constraints.

3. Generate the following reports for analysis:

```
report_timing  
report_area  
report_power
```

3. Physical Design using Cadence Innovus

Physical design converts the gate-level netlist into a layout that can be fabricated on silicon. This step includes floorplanning, placement, clock tree synthesis, and routing.

Inputs:

- Gate-Level Netlist (from Genus)
- Timing Constraints (.sdc)
- Technology Libraries (.lib, .lef)

Outputs:

- Final GDSII Layout
- Post-layout Reports (Timing, Power, Area)

Procedure:

1. Launch Innovus:

```
innovus &
```

2. Import the synthesized netlist and library files.

3. Perform the following steps sequentially:

- **Floorplanning:** Define core and I/O placement regions.
- **Power Planning:** Add power rings and connect VDD/VSS rails.
- **Placement:** Place standard cells efficiently to minimize wire length.
- **Clock Tree Synthesis (CTS):** Distribute clock signals evenly.
- **Routing:** Connect signal nets using NanoRoute.

4. Verify the design through DRC and LVS checks.

5. Export the final GDSII layout:

```
saveDesign ahb_lite_mem_ctrl.enc  
saveNetlist ahb_lite_mem_ctrl.v  
saveGDS ahb_lite_mem_ctrl.gds
```

The completed flow confirms a fully verified and implementable layout for the AHB-Lite Memory Controller.

7. Results

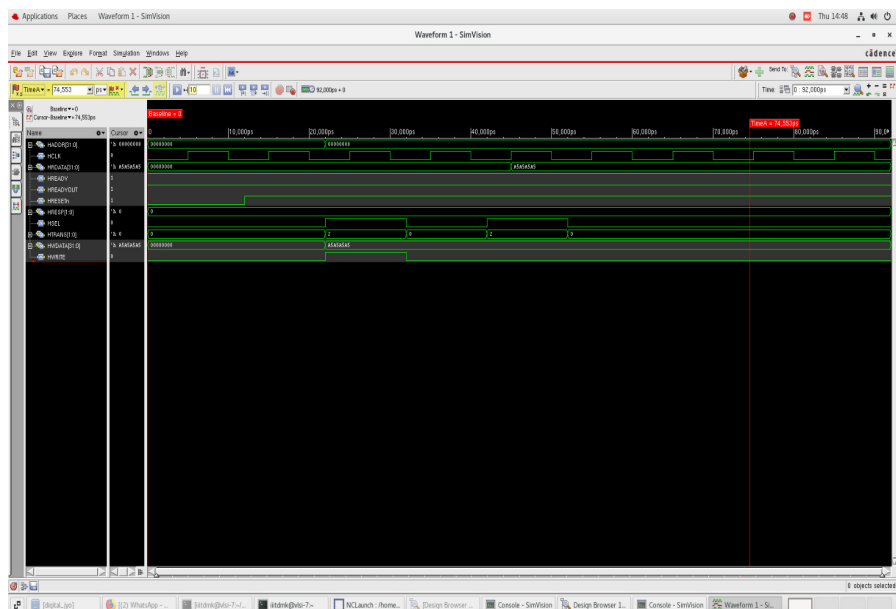


Figure 4: Simulation waveform showing successful read/write operations.

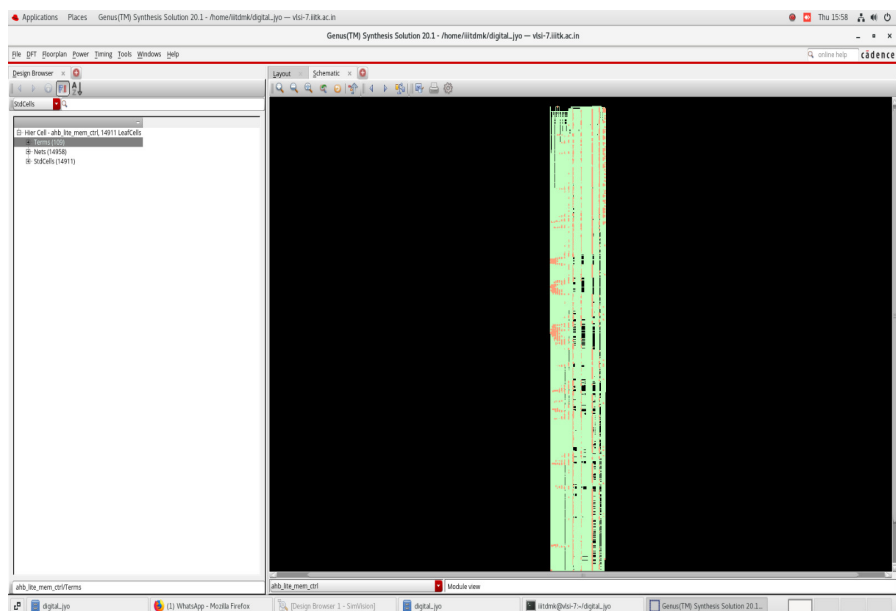


Figure 5: Schematic generated using Cadence genus.

```
@genus:root: 3> report_area
```

```
=====
```

```
Generated by:      Genus(TM) Synthesis Solution 20.11-s111_1
Generated on:      Oct 30 2025  04:22:14 pm
Module:           ahb_lite_mem_ctrl
Operating conditions: slow (balanced_tree)
Wireload mode:    enclosed
Area mode:        timing library
```

```
=====
```

Instance	Module	Cell Count	Cell Area	Net Area	Total Area	Wireload
ahb_lite_mem_ctrl		14911	210548.387	0.000	210548.387	<none> (D)

```
(D) = wireload is default in technology library
```

Figure 6: Synthesis Report showing Total Cell Area obtained from Cadence Genus.

```
iiitdmk@visi-7:~/digital_jyo
```

```
File Edit View Search Terminal Help
```

```
)
(D) = wireload is default in technology library
@genus:root: 4> report_power
Info : Joules engine is used. [RPT-16]
      : Joules engine is being used for the command report_power.
Instance: /ahb_lite_mem_ctrl
Power Unit: W
PDB Frames: /stim#0/frame#0
```

Category	Leakage	Internal	Switching	Total	Row%
memory	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
register	1.12031e-03	1.15721e-02	1.54062e-04	1.28465e-02	93.96%
latch	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
logic	1.64779e-04	4.63800e-04	1.97317e-04	8.25895e-04	6.04%
bbox	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
clock	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
pad	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
pm	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
Subtotal	1.28509e-03	1.20359e-02	3.51379e-04	1.36724e-02	100.00%
Percentage	9.40%	88.03%	2.57%	100.00%	100.00%

```
@genus:root: 5>
```

Figure 7: Power Analysis Report showing Total Dynamic and Leakage Power.

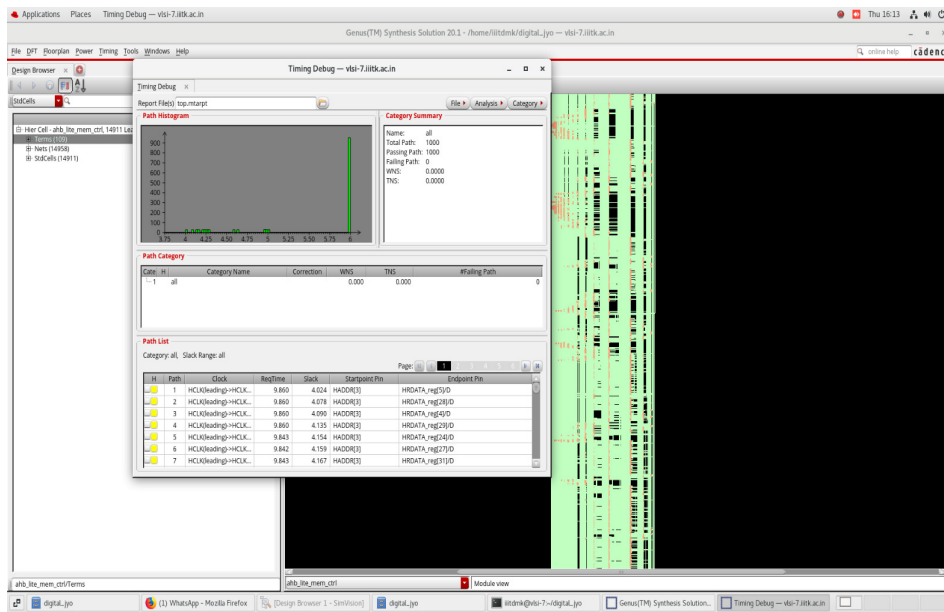


Figure 8: Timing Report generated after synthesis showing slack and critical path delay.

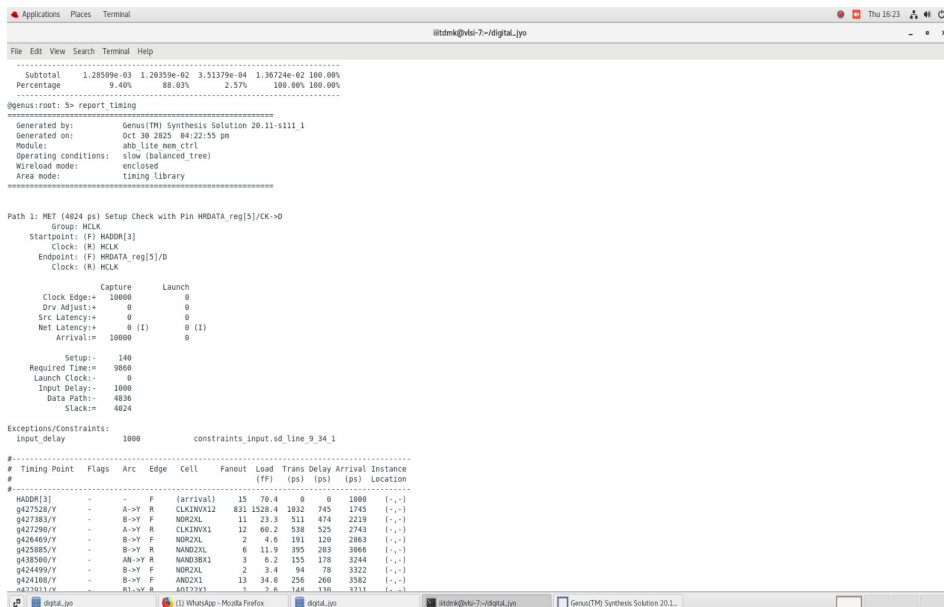


Figure 9: Timing Graph illustrating critical path delay across cells.

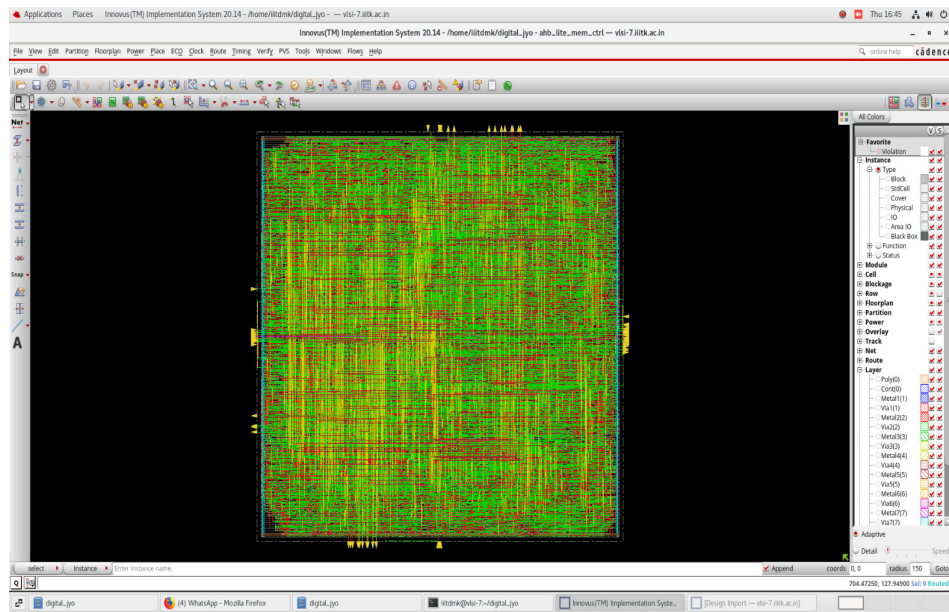


Figure 10: 2D Physical Layout view of the placed and routed design in Cadence Innovus.

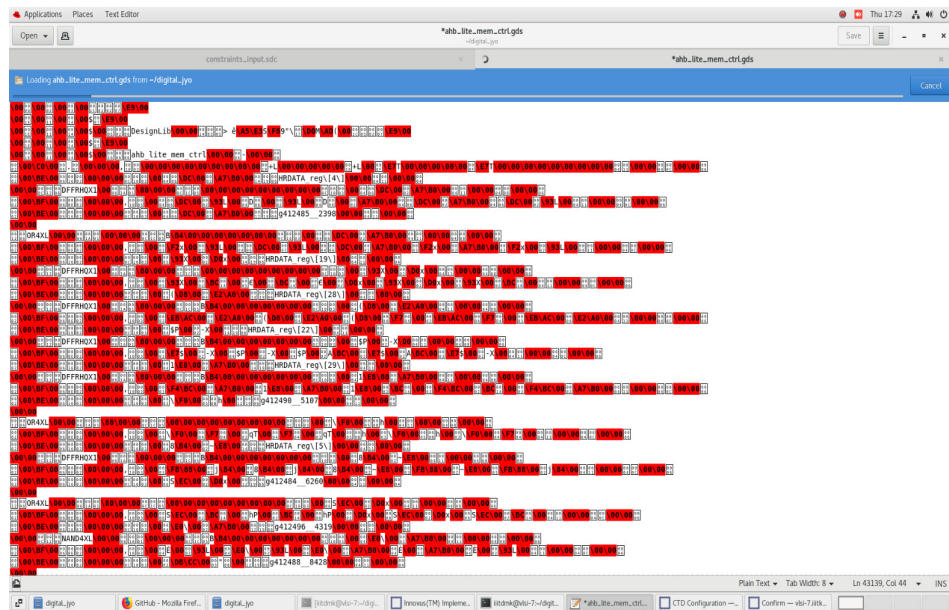


Figure 11: Final GDSII layout generated and exported from Cadence Innovus.

8. Conclusion

The AHB-Lite memory controller was successfully designed and verified using Verilog HDL. Functional verification confirmed proper read/write operations. Physical design using Cadence Genus and Innovus resulted in a clean layout passing DRC and LVS checks, demonstrating a complete RTL-to-GDSII flow.